National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Faculty of Computing

Department of Computing

Laboratory Manual

CS-250 : Data Structure and Algorithms

Class: BESE 16AB • Semester: Fall 2026

Experiment No. 7

Implementation of Stacks in Different Problems

Date	23 October 2026
Time	09:00 AM–12:00 PM & 2:30 PM–5:00 PM
Instructor(s)	Dr. Faisal Shafait
Lab Engineer	Hamza Fayyaz
Mapped CLO	**CLO2 : Solve a given real time problem by applying appropriate data structure and algorithm.** **CLO3 : Practice programs using the latest IDEs ensuring testing, documentation and packaging of programs as per standards practices applicable to the software industry.** **CLO4 : Demonstrate commitment to professional ethics by following engineering norms applicable to the software industry.**:

1 Introduction

In this lab, we will study stacks, a fundamental linear data structure that follows the Last In, First Out (LIFO) principle. Students will use stack operations to solve expression validation, expression conversion and evaluation, and the Tower of Hanoi problem.

We will review stack implementations in C++ and apply push, pop, peek/top, isEmpty, and isFull operations. The exercises emphasize correct stack behavior, CSV file handling, tokenization using `vector<string>`, and object-oriented programming.

2 Objectives

The objective of this lab is to enable students to understand and implement stack ADTs in C++, and apply stacks to balanced-bracket checking, infix-to-postfix conversion, postfix evaluation, and the Tower of Hanoi.

3 Tools and Software Requirements

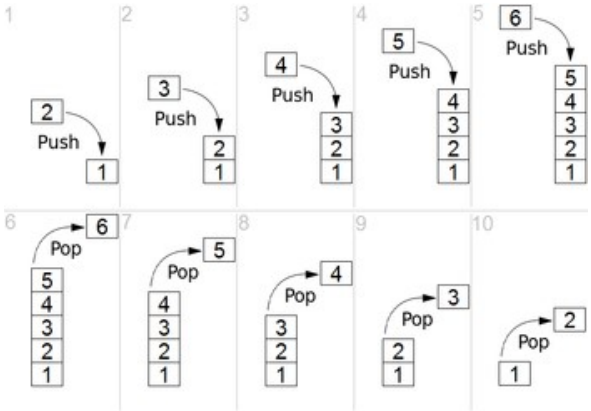
Software: Visual Studio C++

Helping Material: Lecture slides, textbook

4 Description

Stacks

A stack is a container of objects that are inserted and removed according to the last-in first-out (LIFO) principle. Elements can be added and removed only at the top. **Push** adds an item to the top; **pop** removes the top item.



Basic Operations of a Stack

- **Push:** Add an element to the top of the stack.
- **Pop:** Remove the element from the top of the stack.
- **Peek/Top:** View the top element without removing it.
- **IsEmpty:** Check whether the stack is empty.
- **IsFull:** For an array-based implementation, check whether the stack is full.

Example: Array-Based Stack in C++

The following example illustrates a fixed-size array stack, including overflow and underflow checks.

Listing 1. *Stack Implementation*

```cpp
#include <iostream>
using namespace std;
#define MAX 5
class Stack {
private:
    int arr[MAX];
    int top;
public:
    Stack() { top = -1; }
    void push(int value) {
        if (top == MAX - 1)
            cout << "Stack Overflow! Cannot push " << value << endl;
        else {
            arr[++top] = value;
            cout << value << " pushed into the stack." << endl;
        }
    }
    void pop() {
        if (top == -1)
            cout << "Stack Underflow! Cannot pop." << endl;
        else
            cout << arr[top--] << " popped from the stack." << endl;
    }
    void display() {
        if (top == -1) cout << "Stack is empty." << endl;
        else {
            cout << "Stack elements (bottom to top): ";
            for (int i = 0; i <= top; i++) cout << arr[i] << " ";
            cout << endl;
        }
    }
    void peek() {
        if (top == -1) cout << "Stack is empty. Nothing to peek." << endl;
        else cout << "Top element is: " << arr[top] << endl;
    }
    bool isEmpty() { return top == -1; }
    bool isFull() { return top == MAX - 1; }
};
```

5 Lab Tasks

Task 1: Balanced Braces Checker

Problem Statement: Verify whether an input mathematical expression has properly balanced parentheses, square brackets, and curly braces. Use a stack to track the order of opening and closing brackets.

Requirements:

1. Traverse the expression character by character.
2. Push each opening bracket `(`, `[`, or `{` onto the stack.
3. For each closing bracket `)`, `]`, or `}`, check the stack top. If it matches the corresponding opening bracket, pop it; otherwise report the position of the mismatch.
4. At the end, report success if the stack is empty; otherwise report unmatched opening brackets.

Listing 2. *Example Input*

```
1 + 2 * (3 / 4)
```

```
1 + 2 * [3 * 3 + {4 - 5 (6 (7/8/9) + 10) - 11 + (12*8)] + 14
1 + 2 * [3 * 3 + {4 - 5 (6 (7/8/9) + 10)} - 11 + (12*8) / {13 +13}] + 14
```

Listing 3. *Expected Output*

```
This expression is correct.
This expression is NOT correct. Report the bracket error position.
This expression is correct.
```

Task 2: Expression Processing Using Stacks and CSV Files

Problem Statement: Develop a C++ program that reads a comma-separated infix mathematical expression from a CSV file, parses it into a `vector<string>`, converts it to postfix notation using a stack, and evaluates the resulting postfix expression using another stack. Implement the three parts as separate functions.

Part A: Reading and Parsing the CSV File

1. Create `infix.csv` containing one infix expression on a single line. Separate each operand, operator, and parenthesis with a comma.
2. Read the file and split the line using the comma delimiter into a `vector<string>`.
3. Display the original expression and parsed tokens. Implement `vector<string> readCSV(string filename)`.

Part B: Infix to Postfix Conversion

1. Implement `vector<string> infixToPostfix(vector<string> tokens)` using a stack for operators and parentheses.
2. Support `+`, `-`, `*`, and `/` with correct precedence and left associativity for equal-precedence operators. Handle parentheses.
3. Store the resulting postfix tokens in a `vector<string>`, display them, and save them as comma-separated tokens in `postfix.csv`.

Part C: Postfix Expression Evaluation

1. Implement `double evaluatePostfix(vector<string> tokens)` using the postfix vector from Part B.
2. Process tokens left to right: push numeric operands; for an operator, pop the right operand and then the left operand, calculate, and push the result.
3. Display the final result. Use `double` for arithmetic, including fractional division.

Listing 4. *Example Input: infix.csv*

```
(,12,+,8,),*,3
```

Listing 5. *Expected Output*

```
Infix Expression: (,12,+,8,),*,3
Parsed Tokens: ( 12 + 8 ) * 3
Postfix Expression: 12,8,+,3,*
Evaluated Result: 60
Generated file postfix.csv: 12,8,+,3,*
```

Important Notes: Use `vector<string>` for both expressions; treat multi-digit numbers as single operands. Do not hardcode the expression or result. Use stacks for both algorithms. Assume valid expressions containing binary operators and no division by zero.

Task 3: Tower of Hanoi Using Stacks

Problem Statement: Simulate the Tower of Hanoi puzzle using stacks for three rods. Initially all disks are on the source rod, largest at the bottom. Move all disks to the destination rod while obeying the rules: only one disk at a time, only the top disk can move, and no larger disk may be placed on a smaller one.

Requirements:

1. Use three stacks for rods A (source), B (auxiliary), and C (destination).
2. Initialize rod A with n disks in decreasing order.
3. Compute the minimum number of moves, $2^n - 1$.
4. Move disks using stack operations (push, pop, top).
5. If the number of disks is even, swap the roles of the auxiliary and destination rods in the iterative algorithm.
6. Display every move in the format `Move disk X from Rod Y -> Rod Z`.
7. Finish with all disks on the destination rod.

Listing 6. *Example Input*

```
Number of disks = 3
```

Listing 7. *Expected Output*

```
Move disk 1 from A -> C
Move disk 2 from A -> B
Move disk 1 from C -> B
Move disk 3 from A -> C
Move disk 1 from B -> A
Move disk 2 from B -> C
Move disk 1 from A -> C
```

6 Deliverables

Compile a single Word document containing the C++ code and output screenshots for all three tasks and submit it on LMS. Demonstrate your implementations during the lab. Practice object-oriented programming in C++.

7 Important Note

Practice your knowledge of OOP with C++ when creating a solution.

8 Lab Rubrics

Assessment	Doesn't meet Expectation (1–2)	Meets Expectation (3–4)	Exceeds Expectation (5)	Marks
Software Problem Realization (CLO2)	The Student is unable to understand and outline the problem and doesn't use the relevant method to solve it.	The student requires some guidance to completely comprehend the problem and to differentiate the data structure and algorithm comprehensively.	The student fully understands the given problem, is able to analyze the relevant method to solve it, and develops a detailed program flow.	
Software Tool Usage (CLO3)	The student has no idea on how to use the basic tools of the software. The codes have syntax errors, and parts of the codes are missing. Also, they are unable to imitate the required output.	The student has a limited command on the basic tools of the software and operated it under supervision. The codes are correct in terms of their syntax, however, the program output is not always correct.	The student has full command on various tools available in the software. Furthermore, his/her coding is complete and functional, and the program output is correct. Moreover, they can easily manipulate the code to design a particular solution.	
Ethics and Adherence to Laboratory Safety Rules (CLO4)	The student does not behave according to the professional ethics by following ethical norms applicable to the software industry such as acknowledgement while using publicly available data/code. Disturbs the lab environment, doesn't take care of safety measures, and/or isn't punctual.	The student partially demonstrate their commitment to professional ethics by following ethical norms applicable to the software industry such as referencing and acknowledgement while using publicly available data/code. Exhibits better behavior, works by taking into account the safety measures, and is punctual.	The student clearly express the commitment to professional ethics by following ethical norms applicable to the software industry such as referencing and acknowledgement while using publicly available data/code. Encourages others to maintain lab decorum, and alerts them to follow safety measures.	